

TP1 - Secure Log Analyzer

Étudiant : Mariem Erebih

ID: 22005

Module : Module 7.1 – Programming with Rust

Date : 15 Mai 2026

GitHub : https://github.com/sousou678/tp1_log_analyzer

Milestone 0 – Validation de l’environnement

```
marai@maraiem:~/compose-recipes/rust-venv/workspace$ docker compose up -d
[+] up 1/1
✓ Container rust-labs Started
marai@maraiem:~/compose-recipes/rust-venv/workspace$ docker compose exec rustlab bash
marai@3f270dc0761e:/workspace$ cargo new tp1_log_analyzer
  Creating binary (application) `tp1_log_analyzer` package
note: see more `Cargo.toml` keys and their definitions at https://doc.rust-lang.org/cargo/reference/manifest.html
marai@3f270dc0761e:/workspace$ cd tp1_log_analyzer
marai@3f270dc0761e:/workspace/tp1_log_analyzer$ mkdir samples
marai@3f270dc0761e:/workspace/tp1_log_analyzer$ cd s
samples/ src/
marai@3f270dc0761e:/workspace/tp1_log_analyzer$ cd samples/
marai@3f270dc0761e:/workspace/tp1_log_analyzer/samples$ nano auth_sample.log
marai@3f270dc0761e:/workspace/tp1_log_analyzer/samples$ cd ..
marai@3f270dc0761e:/workspace/tp1_log_analyzer$ cargo run
  Compiling tp1_log_analyzer v0.1.0 (/workspace/tp1_log_analyzer)
  Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.48s
  Running `target/debug/tp1_log_analyzer`
Hello, world!
marai@3f270dc0761e:/workspace/tp1_log_analyzer$
```

```
marai@3f270dc0761e:/workspace/tp1_log_analyzer$ cargo run -- samples/auth_sample.log
  Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.01s
  Running `target/debug/tp1_log_analyzer samples/auth_sample.log`
TP1 Secure Log Analyzer
Input file: samples/auth_sample.log
Summary:
- Total lines read: 18
- Failed login events: 5
- Ignored or malformed lines: 13

Top source IPs:
1. 192.0.2.55 -> 2 failed attempts
2. 198.51.100.23 -> 2 failed attempts
3. 203.0.113.10 -> 1 failed attempts

Top targeted users:
1. root -> 2 failed attempts
2. oracle -> 1 failed attempts
3. admin -> 1 failed attempts
4. backup -> 1 failed attempts
marai@3f270dc0761e:/workspace/tp1_log_analyzer$ cargo run -- samples/auth sample.log
```

Milestone 1 – Repository and README

```
Reinitialized existing Git repository in /workspace/tp1_log_analyzer/.git/
maraim@3f270dc0761e:/workspace/tp1_log_analyzer$ git config --global user.email "mariememed08@gmail.com"
maraim@3f270dc0761e:/workspace/tp1_log_analyzer$ git config --global user.name "sousou678"
maraim@3f270dc0761e:/workspace/tp1_log_analyzer$ nano README.md
maraim@3f270dc0761e:/workspace/tp1_log_analyzer$
```

```
maraim@3f270dc0761e:/workspace/tp1_log_analyzer$ cat README.md
# TP1 - Secure Log Analyzer in Rust

## Objective
This tool is a defensive security utility designed to analyze Linux authentication logs.
It identifies failed SSH login attempts, counts them by IP address and username, and highlights suspicious activity without crashing on m
data.

## Environment Requirements
- Docker & Docker Compose
- Rust Toolchain (Cargo 1.80+)

## Build Command
```bash
cargo build
maraim@3f270dc0761e:/workspace/tp1_log_analyzer$
```

```
maraim@3f270dc0761e:/workspace/tp1_log_analyzer$ ls
Cargo.lock Cargo.toml README.md samples src target
maraim@3f270dc0761e:/workspace/tp1_log_analyzer$ ls -R
.:
Cargo.lock Cargo.toml README.md samples src target

./samples:
auth_sample.log

./src:
main.rs parser.rs stats.rs

./target:
CACHEDIR.TAG debug

./target/debug:
build deps examples incremental tp1_log_analyzer tp1_log_analyzer.d

./target/debug/build:
```

## Milestone 2 – Création du fichier d'exemple (Sample log file)

```
maraim@3f270dc0761e:/workspace/tp1_log_analyzer$ ls
Cargo.lock Cargo.toml README.md samples src target
maraim@3f270dc0761e:/workspace/tp1_log_analyzer$ cd samples/
maraim@3f270dc0761e:/workspace/tp1_log_analyzer/samples$ ls
auth_sample.log
maraim@3f270dc0761e:/workspace/tp1_log_analyzer/samples$
```

```

maraine@3f270dc0761e:/workspace/tp1_log_analyzer/samples$ cat auth_sample.log
Jan 10 08:15:21 srv01 sshd[1001]: Failed password for invalid user admin from
203.0.113.10 port 34567 ssh2
Jan 10 08:16:03 srv01 sshd[1002]: Failed password for root from 198.51.100.23 port
55432 ssh2
Jan 10 08:16:44 srv01 sshd[1003]: Accepted password for student from 192.0.2.15 port
44822 ssh2
Jan 10 08:17:01 srv01 sshd[1004]: Failed password for invalid user test from
203.0.113.10 port 34570 ssh2
Jan 10 08:18:11 srv01 sshd[1005]: Failed password for admin from 203.0.113.10 port
34571 ssh2
Jan 10 08:19:41 srv01 sshd[1006]: Invalid user oracle from 192.0.2.55 port 51200
Jan 10 08:20:12 srv01 sshd[1007]: Failed password for root from 198.51.100.23 port
55433 ssh2
Jan 10 08:21:00 srv01 sshd[1008]: Failed password for invalid user guest from
203.0.113.77 port 60000 ssh2
MALFORMED LINE WITHOUT EXPECTED SSH FIELDS
Jan 10 08:22:11 srv01 sshd[1009]: Failed password for backup from 192.0.2.55 port 51201
ssh2
maraine@3f270dc0761e:/workspace/tp1_log_analyzer/samples$ █

```

À sousou678 (36 minutes ago) In 18

## Milestone 3 – Modèle de données (Data model)

```

maraine@3f270dc0761e:/workspace/tp1_log_analyzer$ cd src/
maraine@3f270dc0761e:/workspace/tp1_log_analyzer/src$ ls
main.rs parser.rs stats.rs
maraine@3f270dc0761e:/workspace/tp1_log_analyzer/src$ █

```

À sousou678 (36 minutes ago) In 18

```

#[derive(Debug, Clone, PartialEq, Eq)]
pub struct FailedLogin {
 pub user: String,
 pub ip: String,
}

#[derive(Debug, Clone, PartialEq, Eq)]
pub enum ParseOutcome {
 Failed(FailedLogin),
 Ignored,
 Malformed,
}

#[allow(clippy::collapsible_if)]
pub fn parse_line(line: &str) -> ParseOutcome {
 let line = line.trim();
 if line.is_empty() {
 return ParseOutcome::Ignored;
 }

 if line.contains("Failed password for") || line.contains("Invalid user") {
 let parts: Vec<&str> = line.split_whitespace().collect();
 let mut user = String::new();
 let mut ip = String::new();

 if let Some(from_pos) = parts.iter().position(|&p| p == "from") {

```

```

if from_pos > 0 {
user = parts[from_pos - 1].to_string();
}
if from_pos + 1 < parts.len() {
ip = parts[from_pos + 1].to_string();
}
} else if line.contains("Invalid user") {
if let Some(user_pos) = parts.iter().position(|&p| p == "user") {
if user_pos + 1 < parts.len() {
user = parts[user_pos + 1].to_string();
}
}
}

if user.is_empty() || ip.is_empty() {
ParseOutcome::Malformed
} else {
ParseOutcome::Failed(FailedLogin { user, ip })
}
} else if line.contains("Accepted password") {
ParseOutcome::Ignored
} else {
ParseOutcome::Malformed
}
}

#[cfg(test)]
mod tests {
use super::*;

#[test]
fn test_failed_login() {
let line =
"Jan 10 08:15:21 srv01 sshd[1001]: Failed password for root from 1.2.3.4 port 123 ssh2";
let outcome = parse_line(line);
if let ParseOutcome::Failed(login) = outcome {
assert_eq!(login.user, "root");
assert_eq!(login.ip, "1.2.3.4");
} else {
panic!("Devrait être un FailedLogin");
}
}

#[test]
fn test_accepted_ignored() {
let line = "Jan 10 08:16:44 srv01 sshd[1003]: Accepted password for student from 192.0.2.15 port 44822 ssh2";
assert_eq!(parse_line(line), ParseOutcome::Ignored);
}

```

```

}

#[test]
fn test_malformed_line() {
let line = "MALFORMED LINE WITHOUT EXPECTED SSH FIELDS";
assert_eq!(parse_line(line), ParseOutcome::Malformed);
}
}

```

```

maraine@3f270dc0761e:/workspace/tp1_log_analyzer/src$ cargo check
 Checking tp1_log_analyzer v0.1.0 (/workspace/tp1_log_analyzer)
 Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.07s
maraine@3f270dc0761e:/workspace/tp1_log_analyzer/src$ git log --oneline
71f01a0 (HEAD -> master) Fix: complete README documentation
f4affbd Final technical version: parser fixed, tests passing, clippy satisfied
f372701 Initial Rust project with README and sample data
maraine@3f270dc0761e:/workspace/tp1_log_analyzer/src$

```

## Milestone 4 – Line parser

```

maraine@3f270dc0761e:/workspace/tp1_log_analyzer/src$ cargo test
 Finished `test` profile [unoptimized + debuginfo] target(s) in 0.01s
 Running unittests src/main.rs (/workspace/tp1_log_analyzer/target/debug/deps/tp1_log_analyzer-0105dbcd3624adc9)

running 3 tests
test parser::tests::test_accepted_ignored ... ok
test parser::tests::test_malformed_line ... ok
test parser::tests::test_failed_login ... ok

test result: ok. 3 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in 0.00s

```

```

#[allow(clippy::collapsible_if)]
pub fn parse_line(line: &str) -> ParseOutcome {
 let line = line.trim();
 if line.is_empty() {
 return ParseOutcome::Ignored;
 }

 // On détecte les échecs de connexion (Failed ou Invalid)
 if line.contains("Failed password for") || line.contains("Invalid user") {
 let parts: Vec<&str> = line.split_whitespace().collect();
 let mut user = String::new();
 let mut ip = String::new();

 // On cherche l'IP après le mot "from"
 if let Some(from_pos) = parts.iter().position(|&p| p == "from") {
 if from_pos > 0 {
 user = parts[from_pos - 1].to_string(); // L'utilisateur est avant "from"
 }
 if from_pos + 1 < parts.len() {

```

```

 ip = parts[from_pos + 1].to_string(); // L'IP est après "from"
 }
}

// Sécurité : si on n'a pas trouvé d'IP ou d'user, la ligne est malformée
if user.is_empty() || ip.is_empty() {
 ParseOutcome::Malformed
} else {
 ParseOutcome::Failed(FailedLogin { user, ip })
}
} else if line.contains("Accepted password") {
 ParseOutcome::Ignored // On ignore les connexions réussies
} else {
 ParseOutcome::Malformed // Tout le reste est considéré comme malformé
}
}

```

## Milestone 5 – Statistics

```

maraine@3f270dc0761e:/workspace/tp1_log_analyzer$ cd src/
maraine@3f270dc0761e:/workspace/tp1_log_analyzer/src$ ls
main.rs parser.rs stats.rs
maraine@3f270dc0761e:/workspace/tp1_log_analyzer/src$ █

```

À souscrire 678 / 36 minutes app. - Lp 18

```

use crate::parser::FailedLogin;
use std::collections::HashMap;

pub fn count_by_ip(events: &[FailedLogin]) -> Vec<(String, usize)> {
 let mut counts = HashMap::new();
 for event in events {
 *counts.entry(event.ip.clone()).or_insert(0) += 1;
 }
 let mut sorted: Vec<_> = counts.into_iter().collect();
 sorted.sort_by(|a, b| b.1.cmp(&a.1)); // Trier par score descendant
 sorted
}

pub fn count_by_user(events: &[FailedLogin]) -> Vec<(String, usize)> {
 let mut counts = HashMap::new();
 for event in events {
 *counts.entry(event.user.clone()).or_insert(0) += 1;
 }
 let mut sorted: Vec<_> = counts.into_iter().collect();
 sorted.sort_by(|a, b| b.1.cmp(&a.1));
 sorted
}

```

## Milestone 6 – Command-line interface

```
mod parser;
mod stats;

use parser::{FailedLogin, ParseOutcome, parse_line};
use std::env;
use std::fs::File;
use std::io::{BufRead, BufReader};
use std::process;

fn main() {
// 1. Lire l'argument (le fichier)
let args: Vec<String> = env::args().collect();
if args.len() < 2 {
eprintln!("Usage: cargo run -- <path_to_log_file>");
process::exit(1);
}

let file_path = &args[1];

// 2. Ouvrir le fichier de manière sécurisée
let file = File::open(file_path).unwrap_or_else(|err| {
eprintln!("Error opening file {}: {}", file_path, err);
process::exit(1);
});

let reader = BufReader::new(file);
let mut failed_events: Vec<FailedLogin> = Vec::new();
let mut total_lines = 0;
let mut ignored_malformed = 0;

// 3. Parser ligne par ligne
for line_result in reader.lines() {
let line = match line_result {
Ok(l) => l,
Err(_) => continue,
};

total_lines += 1;
match parse_line(&line) {
ParseOutcome::Failed(login) => failed_events.push(login),
ParseOutcome::Ignored | ParseOutcome::Malformed => ignored_malformed += 1,
}
}

// 4. Afficher les résultats
```



```
println!("TP1 Secure Log Analyzer");
println!("Input file: {}", file_path);
println!("Summary:");
println!("- Total lines read: {}", total_lines);
println!("- Failed login events: {}", failed_events.len());
println!("- Ignored or malformed lines: {}", ignored_malformed);

println!("\nTop source IPs:");
for (i, (ip, count)) in stats::count_by_ip(&failed_events)
.iter()
.take(5)
.enumerate()
{
println!("{}", i + 1, ip, count);
}

println!("\nTop targeted users:");
for (i, (user, count)) in stats::count_by_user(&failed_events)
.iter()
.take(5)
.enumerate()
{
println!("{}", i + 1, user, count);
}
}
```

```
marc@127.0.0.1: /workspace/tp1_log_analyzer $ cargo run -- samples/auth_sample.log
Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.01s
Running `target/debug/tp1_log_analyzer samples/auth_sample.log`
TP1 Secure Log Analyzer
Input file: samples/auth_sample.log
Summary:
- Total lines read: 18
- Failed login events: 5
- Ignored or malformed lines: 13

Top source IPs:
1. 198.51.100.23 -> 2 failed attempts
2. 192.0.2.55 -> 2 failed attempts
3. 203.0.113.10 -> 1 failed attempts

Top targeted users:
1. root -> 2 failed attempts
2. oracle -> 1 failed attempts
3. backup -> 1 failed attempts
4. admin -> 1 failed attempts
```

## Milestone 7 – Code quality

**cargo fmt --check** : Vérifie que le code suit le guide de style officiel. Comme la commande n'affiche rien chez toi, cela confirme que le formatage est **100% conforme**.

**cargo clippy -- -D warnings** : Analyse le code pour trouver des erreurs logiques ou des optimisations. Le message "Finished" sans texte supplémentaire prouve que ton code est **irréprochable**.

**cargo test** : Lance les tests de sécurité. Tes **3 tests** (`test_accepted_ignored`, `test_failed_login`,

```
maraine@3f270dc0761e:/workspace/tp1_log_analyzer$ cargo fmt --check
maraine@3f270dc0761e:/workspace/tp1_log_analyzer$ cargo clippy -- -D warnings
Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.03s
maraine@3f270dc0761e:/workspace/tp1_log_analyzer$ cargo test
Finished `test` profile [unoptimized + debuginfo] target(s) in 0.03s
Running unittests src/main.rs (target/debug/deps/tp1_log_analyzer-0105dbcd3624adc9)

running 3 tests
test parser::tests::test_accepted_ignored ... ok
test parser::tests::test_failed_login ... ok
test parser::tests::test_malformed_line ... ok

test result: ok. 3 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in 0.00s
```

`test_malformed_line`) sont passés avec succès (**ok**).

## Bonus tasks

```
Variables
LOG_FILE = samples/auth_sample.log

Commande par défaut : analyse du log
run:
cargo run -- $(LOG_FILE)

Lancer tous les tests
test:
cargo test

Vérifier la qualité du code (Formatage + Clippy)
lint:
cargo fmt --check
cargo clippy -- -D warnings

Tout faire d'un coup (Vérification complète avant soumission)
all: lint test run
```

```
maraim@3f270dc0761e:/workspace/tp1_log_analyzer$ make run
cargo run -- samples/auth_sample.log
 Compiling tp1_log_analyzer v0.1.0 (/workspace/tp1_log_analyzer)
 Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.21s
 Running `target/debug/tp1_log_analyzer samples/auth_sample.log`
```

[SECURITY ALERT] IPs with more than 3 failed attempts:

No critical threats detected.

TP1 Secure Log Analyzer

Input file: samples/auth\_sample.log

Summary:

- Total lines read: 18
- Failed login events: 5
- Ignored or malformed lines: 13

Top source IPs:

1. 192.0.2.55 -> 2 failed attempts
2. 198.51.100.23 -> 2 failed attempts
3. 203.0.113.10 -> 1 failed attempts

Top targeted users:

1. root -> 2 failed attempts
2. admin -> 1 failed attempts
3. backup -> 1 failed attempts
4. oracle -> 1 failed attempts